

Project 3 Report — Group10

Muhan Zhang

Danhua Zhao

Team & Repo

- Group: 10
- Members: Muhan Zhang, Danhua Zhao
- Division of work: Muhan Zhang (2PC, Raft core, Docker), Danhua Zhao (testing, logs/screenshots, report)
- Base repo: <https://github.com/J1anXu/Distributed-picture-sharing-system>
- Submission repo (our repo):
<https://github.com/ricolalemon/Distributed-picture-sharing-system-consensus>
- Consensus additions: `consensus_node/`, `docker-compose-consensus.yml`

Overview

Added a 5-node consensus cluster with Two-Phase Commit (vote+decision) and Raft (leader election, heartbeats, log replication). Followers forward client commands to the leader; majority commit persists state to `/data/consensus_store.json`.

Architecture Highlights

- Heartbeat: 1s; Election timeout: randomized 1.5–3s.
- Full-log replication on `AppendEntries`; majority ACK triggers commit.
- Protos: `consensus_node/two_phase.proto`, `consensus_node/raft.proto`.
- Docker: 5 homogeneous nodes; host ports 6001–6005 map to 6000.

Implementation Details

- 2PC: coordinator issues `Vote` to all peers; simple veto rule (keys starting with “deny” abort); pending txns tracked per node; Decision applies commit/abort and writes to KV store; required prints implemented on client/server.
- Raft: follower/candidate/leader roles; randomized election timeout to avoid split vote; leader heartbeats every 1s; full-log replication on `AppendEntries`; majority ACK advances commit index; client commands accepted on any node and forwarded to leader when needed.

- Persistence: per-node JSON at `/data/consensus_store.json` (Docker volume) for committed state.

Logging Formats (per requirements)

- 2PC (client): Phase P of Node N sends RPC R to Phase P of Node M
- 2PC (server): Phase P of Node N runs RPC R called by Phase P of Node M
- Raft (client): Node X sends RPC `<rpc>` to Node Y
- Raft (server): Node X runs RPC `<rpc>` called by Node Y

Container Topology

- Services: `consensus-node1..5`
- Ports: host 6001–6005 → container 6000
- Network: default docker network; all nodes reachable by service name
- Data: volume per node for `/data/consensus_store.json`

How to Run

1. Start:


```
docker compose -f docker-compose-consensus.yml up --build -d
```
2. 2PC demo:


```
grpcurl -plaintext -d '{"key":"k1","value":"v1"}'
localhost:6001 twopc.TwoPhaseService/StartTransaction
```
3. Raft demo (follower OK):


```
grpcurl -plaintext -d '{"command":"set a 1","client_id":"cli"}'
localhost:6003 raft.RaftService/ClientCommand
```
4. Stop:


```
docker compose -f docker-compose-consensus.yml down
```
5. Host helper:


```
PYTHONPATH=. .venv-local/bin/python
consensus_node/demo_raft.py "set a 1"
```
6. In container:


```
PYTHONPATH=/app TARGET=consensus-nodeX:6000
python consensus_node/demo_raft.py "set k v"
```

Validation (Q5 Test Cases)

1. Initial election: RequestVote + AppendEntries.
2. Heartbeat stability: steady AppendEntries, no elections.
3. Log replication: `set a 1` committed on all nodes.
4. Client forwarding: follower forwards ClientCommand to leader; commit visible on all nodes.
5. Leader failover/restart: stop old leader, elect new; restart old leader; post-failover `set c 3` committed everywhere.

Notes

- Compose omits version to silence warnings.
- Use `.venv-local/bin/python` on host; set `PYTHONPATH=/app` inside containers.
- Logs follow required 2PC and Raft RPC formats.
- If `grpc` missing: install deps with `.venv-local/bin/pip install -r requirements.txt`; re-run commands with `PYTHONPATH=. .venv-local/bin/python ...`
- Check state quickly: `docker exec <node> cat /data/consensus_store.json`

Screenshots

[illegible]

Figure 1: Initial election logs

[illegible]

Figure 2: Logs after set a 1

Figure 4: Leader restart logs

```
$ state after set a 1
{"demo-key": "demo-value", "a": "1", "b": "2", "c": "3", "demo": "1"}
```

Figure 5: State after set a 1

```
$ state after set b 2
{"demo-key": "demo-value", "a": "1", "b": "2", "c": "3", "demo": "1"}
```

Figure 6: State after set b 2

```
$ state after set c 3
{"demo-key": "demo-value", "a": "1", "b": "2", "c": "3", "demo": "1"}
```

Figure 7: State after set c 3 (post-failover)